

Continuum Simulator: monitoraggio ambientale e aggregazione distribuita Edge–Cloud

Alessandro Vassallo – Matricola 0366572
Università degli Studi di Roma Tor Vergata
Sistemi Distribuiti e Cloud Computing – A.A. 2025/26

Sommario—Questa relazione presenta un’applicazione distribuita sviluppata in Go per il monitoraggio ambientale nel compute continuum. Il sistema riproduce misure reali derivate da Sensor.Community e distribuisce l’elaborazione tra livelli Edge e Cloud mediante MQTT e Apache Kafka. La soluzione adotta un’aggregazione gerarchica basata su finestre temporali e un livello Cloud scalabile orizzontalmente tramite partizionamento Kafka e consumer group. La valutazione sperimentale analizza il comportamento del sistema al variare del numero di Cloud Worker e delle condizioni di comunicazione.

I. INTRODUZIONE

L’obiettivo del progetto è lo sviluppo di un’applicazione distribuita nel compute continuum per il monitoraggio di misure ambientali, distribuendo l’elaborazione tra livelli Edge e Cloud. Il caso di studio utilizza dati provenienti da sensori BME280 del dataset pubblico Sensor.Community, relativi a temperatura, umidità e pressione e raccolti nel mese di gennaio 2025.

Il dataset storico viene utilizzato per simulare un sistema di monitoraggio continuo. Le osservazioni vengono riprodotte preservandone le distanze temporali in event time mediante un replay accelerato e inviate ai rispettivi Edge Site. Ciascun Edge esegue una prima fase di validazione e aggregazione locale, mentre il livello Cloud combina successivamente i risultati provenienti dalle diverse zone per ottenere una vista complessiva del sistema. Questa suddivisione permette di distribuire la computazione lungo il continuum e di ridurre la quantità di dati che deve essere trasferita verso il Cloud rispetto all’invio di ogni singola osservazione.

Poiché il progetto è svolto individualmente, tra i requisiti alternativi previsti dalla traccia è stata scelta la scalabilità orizzontale. La valutazione sperimentale studia quindi il comportamento del sistema al variare del numero di Cloud Worker, mantenendo invariato il workload, e analizza inoltre l’impatto delle condizioni di rete sulla pipeline distribuita. L’applicazione è sviluppata interamente in Go, containerizzata tramite Docker e distribuita su AWS mediante un processo di deployment automatizzato.

II. DATASET E COSTRUZIONE DEL WORKLOAD

Lo scenario è stato costruito a partire dal dataset relativo ai sensori ambientali BME280. È stata considerata la finestra temporale di gennaio 2025 e il dataset, di dimensione pari a circa 6.36 GB, è stato analizzato in modo incrementale per evitare il caricamento completo in memoria.

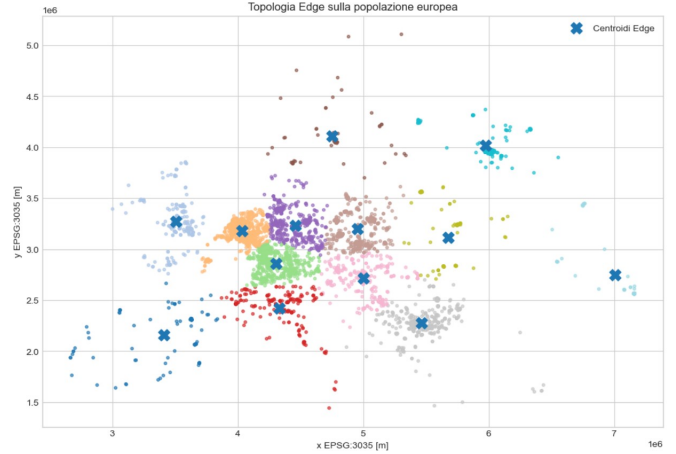


Figura 1. Topologia geografica della popolazione europea e centroidi dei 13 Edge Site.

Una prima fase di preprocessing ha permesso di verificare la qualità dei dati, filtrare le osservazioni esterne alla finestra temporale e individuare i sensori dotati di coordinate geografiche valide. Delle variabili disponibili, le coordinate geografiche sono state utilizzate per la costruzione della topologia, mentre temperatura, umidità e pressione sono state mantenute come misure da riprodurre durante l’esecuzione. Le variabili relative all’altitudine e alla pressione al livello del mare, completamente prive di valori, sono state escluse.

Lo studio è stato limitato all’area europea. Per ogni sensore è stata determinata una posizione rappresentativa e le coordinate geografiche WGS84 sono state proiettate nel sistema metrico EPSG:3035, in modo da applicare il clustering su distanze spaziali coerenti. La popolazione risultante comprende 5102 sensori europei localizzabili.

La suddivisione in Edge Site è stata ottenuta tramite K-Means sulle coordinate proiettate. Sono stati valutati valori di k compresi tra 5 e 15 utilizzando il Silhouette Score come criterio principale di selezione, affiancato dall’inertia e dal raggio massimo dei cluster come indicatori diagnostici. La configurazione selezionata è risultata $k = 13$, con Silhouette Score pari a circa 0.521; l’elbow method individua invece $k = 9$, utilizzato solamente come riferimento diagnostico.

Una volta determinata la topologia sull’intera popolazione europea, è stato costruito il workload sperimentale selezionando 150 sensori. I sensori sono stati distribuiti nel modo più

uniforme possibile tra i 13 Edge Site (11–12 sensori per Edge). La selezione all'interno di ciascun Edge è stata effettuata utilizzando un seed pseudo-casuale fissato. In questo modo ogni Edge partecipa in maniera comparabile agli esperimenti, pur mantenendo le differenze naturali nel tasso di generazione degli eventi.

Infine, la fase di preprocessing produce due tipi di artefatti utilizzati durante l'esecuzione: il file rappresentante la topologia, che associa i sensori selezionati agli Edge Site, e 13 shard di replay, uno per ciascun Edge, contenenti identificativo del sensore, timestamp, pressione, temperatura e umidità.

III. ARCHITETTURA DEL SISTEMA

L'architettura realizzata è una pipeline distribuita nel compute continuum, organizzata tra livelli Edge e Cloud come mostrato in Fig. 2. Il sistema riproduce le osservazioni dei sensori selezionati dal dataset storico, le elabora localmente negli Edge Site e propaga verso il Cloud risultati progressivamente aggregati.

La pipeline è composta dai seguenti stadi logici.

- 1) **Simulator:** Per ciascuno dei 13 Edge Site è presente un Simulator che riproduce gli eventi dei sensori assegnati preservandone le distanze temporali in event time, opportunamente accelerate. La telemetria viene pubblicata tramite MQTT verso il broker Mosquitto associato al sito.
- 2) **Elaborazione Edge:** Ogni Edge Site comprende un broker MQTT e un Edge Gateway. Il gateway riceve e valida le misure di temperatura, umidità e pressione e le aggrega localmente su finestre temporali di durata configurabile. Questa elaborazione consente di ridurre il volume di dati inoltrato verso il Cloud, trasferendo statistiche aggregate anziché le singole osservazioni.
- 3) **Trasporto Edge-Cloud:** Gli aggregati prodotti dagli Edge vengono pubblicati sul topic `Kafka edge-aggregates`. Kafka è utilizzato per disaccoppiare produttori e consumatori e per fornire persistenza e ordinamento dei record all'interno delle partizioni [3]. Gli Edge sono distribuiti staticamente tra un numero configurabile di partizioni. L'assegnamento è calcolato prima del deployment sulla base del numero di eventi dei replay shard, così da ridurre lo sbilanciamento atteso senza introdurre decisioni dinamiche nel data path.
- 4) **Elaborazione Cloud.** I Cloud Worker appartengono allo stesso consumer group Kafka. Ciascuna partizione è assegnata a un solo worker e conserva uno stato di aggregazione indipendente. Lo stato è separato logicamente per partizione, ma risiede in memoria presso la replica assegnataria e non viene migrato: l'assegnamento deve pertanto restare stabile per l'intera esecuzione. I contributi degli Edge vengono combinati su finestre temporali di durata configurabile, definita come multiplo della finestra Edge. Aumentando il numero di worker è possibile elaborare più partizioni in parallelo, fino al limite imposto dal numero di partizioni del topic.

- 5) **Aggregazione globale e persistenza:** I risultati dei Cloud Worker vengono pubblicati sul topic `cloud-partition-aggregates` e consumati dal Global Aggregator. Quest'ultimo combina i contributi relativi allo stesso intervallo temporale provenienti dalle diverse partizioni e produce un unico risultato globale, successivamente scritto su log oppure persistito in PostgreSQL.

Il numero di Cloud Worker può essere modificato tra esecuzioni senza cambiare la suddivisione logica del carico né il significato dei risultati prodotti. Il numero di partizioni Kafka e il numero di repliche del microservizio Cloud Worker sono configurabili separatamente. Le partizioni determinano la granularità con cui il workload viene suddiviso e pongono un limite superiore al parallelismo ottenibile dal consumer group, mentre il numero di Cloud Worker determina quanta parte di tale parallelismo viene effettivamente sfruttata. Aumentando il numero di repliche, le stesse partizioni vengono distribuite tra un numero maggiore di worker, senza modificare il workload né la suddivisione logica dei dati. Questa separazione consente di valutare la scalabilità orizzontale del livello Cloud mantenendo invariati gli input e i risultati attesi.

IV. IMPLEMENTAZIONE

A. Aggregazione Edge e Cloud

L'elaborazione utilizza finestre tumbling basate sull'event time. Ciascun Edge mantiene una finestra corrente e assegna ogni evento alla finestra determinata dal relativo timestamp. Per ogni grandezza ambientale vengono mantenuti il numero di valori validi e non validi, la somma, il minimo e il massimo. Al momento della chiusura della finestra viene inoltre calcolata la media sui soli valori validi.

Le statistiche sono state scelte in modo da poter essere combinate nei livelli successivi senza conservare le singole osservazioni. Per ottenere un aggregato a partire da più contributi, vengono sommati il numero di valori validi e le rispettive somme, mentre il minimo e il massimo sono ricavati considerando i valori estremi dei contributi che contengono almeno un'osservazione valida. La media viene quindi calcolata a partire dalla somma complessiva e dal numero totale di valori validi.

In questo modo la media globale viene ottenuta pesando implicitamente ciascun contributo in base al numero di osservazioni che contiene, evitando di calcolare una semplice media delle medie locali, che sarebbe errata quando gli aggregati hanno numerosità differenti.

Le finestre Cloud hanno durata configurabile, ma devono essere un multiplo intero della durata delle finestre Edge. Ogni `EdgeAggregate` appartiene quindi interamente a una sola finestra Cloud. I Cloud Worker combinano gli aggregati ricevuti per partizione Kafka e producono un `CloudPartitionAggregate`; il Global Aggregator applica infine la stessa operazione di riduzione ai contributi delle diverse partizioni relativi allo stesso intervallo temporale.

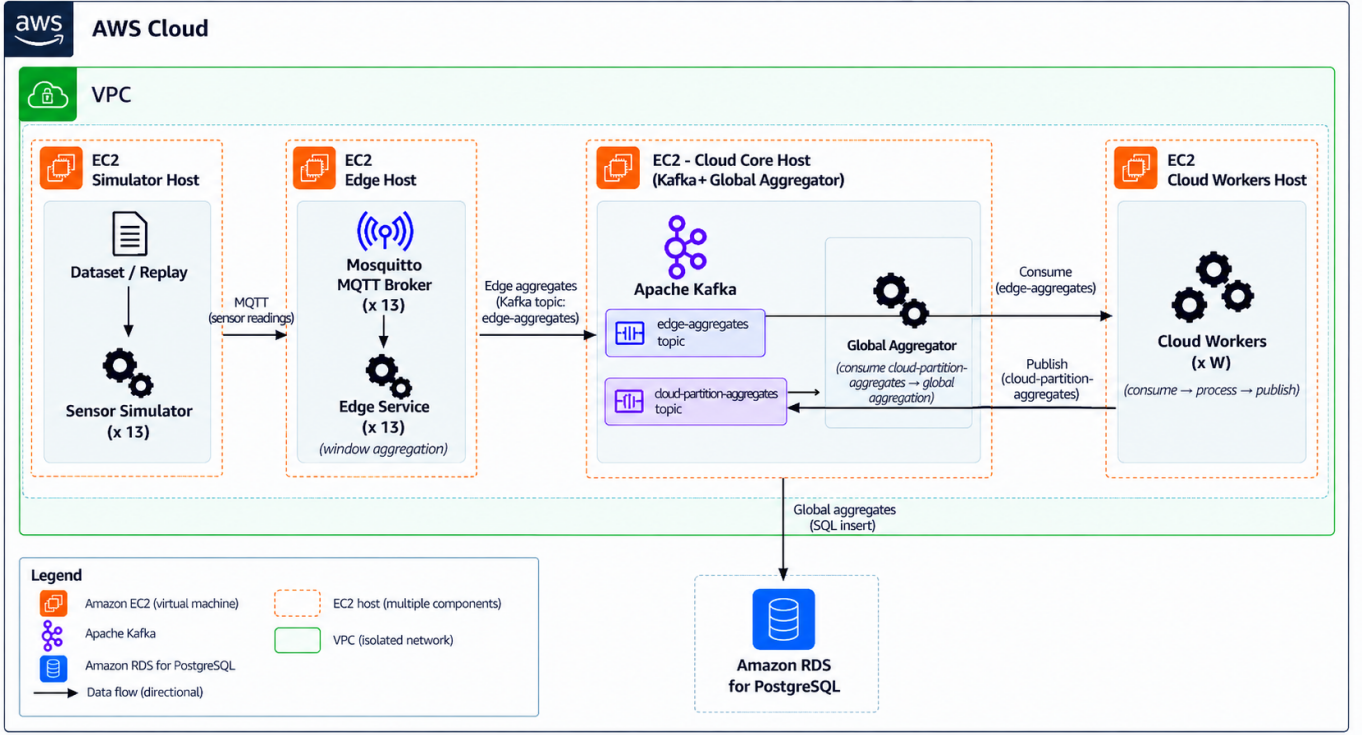


Figura 2. Deployment distribuito su AWS e flusso dei dati tra Simulator, Edge, Kafka, Cloud Worker, Global Aggregator e PostgreSQL. Amazon RDS appartiene alla stessa VPC ed è rappresentato separatamente nella parte inferiore esclusivamente per leggibilità.

B. Progresso event-time e completamento

La chiusura delle finestre Cloud è governata dal progresso in event time delle sorgenti e non da timeout in processing time. Ogni `EdgeAggregate` contiene il campo `CompleteThrough`, che rappresenta il progresso in event time del relativo Edge e svolge un ruolo analogo a un watermark per sorgente: certifica che l'Edge non produrrà più aggregati con estremo superiore precedente o uguale a tale valore.

Quando l'arrivo di un evento determina il passaggio a una nuova finestra, l'Edge emette l'aggregato della finestra precedente e può avanzare `CompleteThrough` fino all'inizio della nuova finestra. In questo modo vengono certificate anche eventuali finestre intermedie prive di eventi, per le quali gli Edge non producono aggregati espliciti.

Il Cloud mantiene separatamente il progresso di ciascun Edge appartenente alla partizione Kafka elaborata e combina tali informazioni per determinare la frontiera di avanzamento della partizione, utilizzata per stabilire quando una finestra Cloud può essere considerata completa. Indicando con C_e il valore `CompleteThrough` dell'Edge attivo e e con S il massimo disallineamento ammesso tra le sorgenti, la frontiera è determinata da

$$F_P = \max \left(\min_{e \in E_P} C_e, \max_{e \in E_P} C_e - S \right). \quad (1)$$

Finché almeno un Edge attivo non ha ancora comunicato il proprio progresso, la frontiera non può avanzare. In condizioni normali essa coincide con il minimo progresso degli Edge della partizione; il secondo termine impedisce invece che una sorgente eccessivamente in ritardo blocchi indefinitamente la finalizzazione delle finestre.

Una finestra Cloud viene emessa quando il proprio estremo superiore non è successivo a F_P . Negli esperimenti principali S viene mantenuto sufficientemente ampio da non forzare l'avanzamento della frontiera: la completezza dipende quindi dal progresso naturale di tutti gli Edge della partizione.

Un aggregato relativo a una finestra già superata dalla frontiera è considerato *late*: non modifica né riapre il risultato già emesso. Il progresso contenuto nel record può comunque essere utilizzato per aggiornare lo stato dell'Edge e consentirgli di riallinearsi alle altre sorgenti.

Poiché il workload è ottenuto dal replay di un dataset finito, è inoltre necessario determinare esplicitamente la conclusione del flusso. Al termine del proprio replay ciascun Simulator invia un marker di fine input al rispettivo Edge. L'Edge pubblica l'ultima finestra ancora aperta e, successivamente, un `EdgeEndOfInput`. Quando tutti gli Edge appartenenti a una partizione sono terminati, il Cloud Worker emette gli eventuali aggregati ancora pendenti e infine un `PartitionEndOfReplay`.

Il Global Aggregator utilizza il campo `CompleteThrough` contenuto nei `CloudPartitionAggregate`, insieme ai

marker terminali delle partizioni, per distinguere un contributo non ancora arrivato da un contributo effettivamente assente. Il progresso globale è calcolato con la stessa politica bounded applicata dal Cloud agli Edge. Indicando con C_p il progresso della partizione sorgente attiva p e con S_G il massimo disallineamento ammesso, la frontiera globale è

$$F_G = \max \left(\min_{p \in P_A} C_p, \max_{p \in P_A} C_p - S_G \right), \quad (2)$$

dove P_A è l'insieme delle partizioni non ancora terminate. Finché una partizione attiva non ha comunicato alcun progresso, la frontiera globale non avanza. Una finestra globale può essere prodotta quando, per ogni partizione sorgente, è disponibile il relativo aggregato oppure il progresso, la frontiera globale o il marker terminale certificano che non ne verranno prodotti altri per quell'intervallo. Un contributo che raggiunge il Global dopo che la relativa finestra è stata superata da F_G viene registrato come *late* e non riapre il risultato già emesso. Il replay termina quando tutte le partizioni hanno raggiunto lo stato terminale e non rimangono finestre pendenti.

C. Comunicazione e affidabilità

1) *MQTT tra Simulator ed Edge*: La comunicazione tra Simulator ed Edge utilizza MQTT. Gli eventi dei sensori sono serializzati in JSON e pubblicati con QoS 0. JSON è stato scelto in questo tratto della pipeline per la semplicità della rappresentazione e per la facilità con cui i messaggi possono essere ispezionati durante lo sviluppo e il debugging.

La telemetria viene prodotta con frequenza elevata: l'utilizzo di QoS 0 evita di attendere una conferma per ogni singola osservazione e permette al replay di procedere secondo il rate configurato. Il marker di fine replay utilizza invece QoS 1 e viene considerato completato soltanto dopo la ricezione del relativo PUBACK, poiché la sua perdita impedirebbe di determinare correttamente la terminazione della sorgente.

2) *Kafka tra Edge e Cloud*: La comunicazione tra Edge e Cloud e tra Cloud Worker e Global Aggregator è affidata ad Apache Kafka. In questo tratto i messaggi sono serializzati in formato Apache Avro. Rispetto al JSON, Avro fornisce una rappresentazione binaria più compatta e definisce esplicitamente la struttura e i tipi dei record scambiati, caratteristica utile per aggregati composti da numerosi campi numerici e temporali. Gli schemi Avro sono inclusi direttamente nell'applicazione e devono quindi essere condivisi in modo coerente da producer e consumer; non viene utilizzato uno Schema Registry.

Gli Edge utilizzano writer Kafka sincroni e attendono l'acknowledgment delle scritture prima di proseguire. Gli aggregati appartenenti allo stesso Edge e il relativo `EdgeEndOfInput` sono scritti sulla medesima partizione; prima di pubblicare il marker terminale vengono completate le scritture degli aggregati precedenti.

Analogamente, il Cloud pubblica per ciascuna partizione gli eventuali `CloudPartitionAggregate`, che includono nel campo `CompleteThrough` l'informazione di progresso della partizione, e successivamente, al termine del replay, il

relativo `PartitionEndOfReplay`. Tale ordinamento garantisce che il consumer downstream riceva i dati prima delle informazioni che ne certificano il completamento.

I Cloud Worker effettuano il commit degli offset Kafka solamente dopo che il record di input è stato elaborato e gli eventuali output sono stati pubblicati con successo. I commit possono essere raggruppati in batch configurabili per ridurre il costo; la ricezione di un marker terminale forza invece il commit degli offset ancora pendenti. Se un worker gestisce più partizioni, conserva per ciascuna il massimo next offset elaborato e include tutti gli avanzamenti pendenti nel commit.

Il Global Aggregator applica lo stesso ordine, effettuando per ciascun record il commit dopo l'elaborazione e l'eventuale scrittura del risultato nel sink; il batching configurabile degli offset riguarda invece i Cloud Worker. Questa strategia evita di confermare un record prima che i suoi effetti siano stati propagati, ma non realizza una transazione atomica end-to-end tra stato, output Kafka e offset di consumo.

Un arresto dopo la pubblicazione dell'output ma prima del commit può quindi causare la riletture di un record. Per questo motivo gli aggregati utilizzano identificativi deterministici e i componenti riconoscono i duplicati identici, mentre nel sink PostgreSQL l'identificativo dell'aggregato è una chiave univoca.

D. Deployment e piattaforma software

L'applicazione è sviluppata in Go e ciascun componente principale viene distribuito come container Docker. La configurazione del sistema è descritta tramite file YAML, dai quali il programma `deploygen` genera automaticamente i file Docker Compose necessari all'esecuzione. In questo modo parametri quali numero di Cloud Worker, numero di partizioni Kafka, dimensione delle finestre e parametri di batching possono essere modificati senza intervenire sul codice applicativo.

Per il deployment distribuito l'infrastruttura AWS è definita mediante Terraform. Sono previsti quattro host EC2 con ruoli distinti: Simulator, Edge, Cloud Core e Workers. L'host Simulator esegue i processi di replay, l'host Edge ospita i broker Mosquitto e gli Edge Gateway, l'host Workers esegue le repliche dei Cloud Worker, mentre Cloud Core ospita Kafka e il Global Aggregator. La separazione consente di far attraversare realmente la rete ai flussi Simulator-Edge, Edge-Cloud e Cloud Worker-Global Aggregator e di modificare indipendentemente il numero di worker utilizzato negli esperimenti.

La persistenza dei risultati globali può essere affidata a un'istanza PostgreSQL gestita tramite Amazon RDS. Il database non è esposto pubblicamente ed è raggiungibile dal solo Cloud Core. L'infrastruttura, la configurazione applicativa e l'avvio degli esperimenti sono gestiti tramite script dedicati, in modo da rendere ripetibile la preparazione dell'ambiente e la raccolta degli artefatti prodotti da ogni esecuzione. Prima di ogni run che utilizza questo sink, il runner verifica PostgreSQL ed esegue esplicitamente il `TRUNCATE` della tabella

Tabella I
CONFIGURAZIONE COMUNE DELLE CAMPAGNE SPERIMENTALI.

Parametro	Valore
Edge Site / Simulator	13
Partizioni edge-aggregates	6
Finestra Edge	5 min
Finestra Cloud	15 min
Fattore di accelerazione	20 000
Batch producer Edge	10
Batch commit Cloud	100

global_aggregates, così che essa contenga soltanto le finestre finali dell'esecuzione corrente.

Le principali librerie utilizzate dall'applicazione sono Eclipse Paho per MQTT, kafka-go per la comunicazione con Kafka, goavro per la serializzazione Avro e pgx per l'accesso a PostgreSQL. Il preprocessing offline del dataset è invece realizzato in Python mediante pandas, scikit-learn e pyproj. Non viene utilizzato un framework esterno di stream processing: la gestione delle finestre, dello stato e del progresso event-time è implementata direttamente nei componenti Go.

V. VALUTAZIONE SPERIMENTALE

A. Ambiente e metodologia sperimentale

La valutazione risponde a due domande. La prima riguarda la capacità del livello Cloud di elaborare lo stesso workload riducendo il ritardo di produzione dei risultati all'aumentare delle repliche. La seconda riguarda il compromesso tra latenza e completezza quando una sorgente logica viene rallentata sui collegamenti Edge-Cloud e Cloud Worker-Global Aggregator. Le prove sono eseguite sul deployment AWS di Fig. 2.

Il workload definitivo comprende 2 106 784 osservazioni distribuite tra 13 replay shard. Il replay usa un fattore di accelerazione pari a 20 000; tale valore è stato scelto mediante una calibrazione preliminare per esercitare la pipeline senza introdurre intenzionalmente perdite nei componenti a monte. La configurazione comune alle campagne è riassunta in Tab. I.

Per ogni run vengono conservati configurazione effettiva, log, metriche dei container, serie temporali del consumer lag e aggregati finali PostgreSQL. La riconciliazione verifica gli eventi offerti dai Simulator, quelli accettati e processati dagli Edge, gli eventi presenti nei risultati globali e quelli eventualmente scartati come late. Vengono inoltre controllati errori di pubblicazione, duplicati, finestre incomplete, terminazione del protocollo e lag finale.

Nella campagna di scalabilità una run è ammessa al confronto prestazionale soltanto se produce il 100% degli eventi, senza late record o perdite a monte. Negli esperimenti bounded, invece, i late record sono un effetto intenzionale della politica studiata e non rendono da soli non valida la run: devono essere contabilizzati e spiegare interamente la differenza tra eventi elaborati e risultato globale. Restano condizioni necessarie l'assenza di drop non intenzionali, la corretta terminazione e il lag finale nullo.

La metrica principale è la *global emission latency*, cioè il ritardo con cui una finestra globale viene emessa rispetto all'i-

Tabella II
RISULTATI DELLA CAMPAGNA DI SCALABILITÀ. LATENZE E DURATA SONO ESPRESSE IN SECONDI.

W	p50	p95	p99	Durata
1	0.57	1.21	1.40	125.82
2	0.39	1.08	1.60	125.53
3	0.39	0.83	1.16	125.54
4	0.34	0.83	1.10	125.45
6	0.35	1.01	1.58	126.18

stante nominale previsto dalla timeline accelerata. Indicando con t_0 l'avvio sincronizzato del replay, con e_0 l'origine della timeline del dataset, con f il fattore di accelerazione e con w_e la fine della finestra, la deadline nominale è

$$d(w_e) = t_0 + \frac{w_e - e_0}{f}. \quad (3)$$

La latenza è la differenza tra l'istante di emissione del Global Aggregator e $d(w_e)$. L'istante di emissione viene registrato prima del completamento della scrittura PostgreSQL e non rappresenta quindi la latenza percepita da un eventuale servizio di interrogazione. Vengono riportati mediana, 95-esimo e 99-esimo percentile, insieme al consumer lag, all'utilizzo di CPU e memoria e al tempo complessivo della run. Poiché il replay impone il ritmo di ingresso, makespan e goodput sono utilizzati soprattutto come controlli di capacità e completezza.

Il lag Kafka è calcolato rispetto agli offset committati. Il batch di commit dei Cloud Worker può pertanto farvi comparire record già elaborati ma non ancora confermati, fino alla successiva operazione di commit. Il valore è usato con le serie temporali e con il lag finale, senza interpretarne automaticamente il massimo come backlog applicativo effettivo.

B. Scalabilità del livello Cloud

La campagna usa un fixed workload e varia il numero di Cloud Worker tra $W \in \{1, 2, 3, 4, 6\}$. Il topic mantiene sei partizioni in tutte le prove: in questo modo cambia soltanto il numero di partizioni elaborate in parallelo, mentre restano invariati dati, assegnamento degli Edge e risultati attesi. Sei worker costituiscono il limite di parallelismo utile della configurazione; repliche ulteriori rimarrebbero senza partizioni assegnate. Ogni configurazione è stata eseguita due volte.

Tutte le dieci run hanno superato i controlli di qualità: ciascuna ha elaborato 2 106 784 eventi, prodotto 2 710 finestre globali complete, senza late record, duplicati o errori di protocollo, e terminato con lag nullo. La Tab. II riporta la media delle due repliche.

Il miglioramento è evidente fino a quattro worker: rispetto a W1, W4 riduce la mediana di circa il 40% e il p95 di circa il 32%. W6 non offre un ulteriore beneficio e mostra una lieve regressione dei percentili di coda. Il makespan rimane sostanzialmente invariato, come atteso per un workload il cui ritmo è determinato dal replay. I risultati mostrano quindi un beneficio del parallelismo sulla disponibilità delle finestre, ma

anche rendimenti decrescenti prima del limite teorico di sei worker.

Il massimo lag committato cresce da 114 record con W1 a 459 con W6, pur ritornando sempre a zero. Tale crescita non indica da sola un accumulo di lavoro: con più consumer indipendenti aumentano anche i batch di offset pendenti osservabili. Inoltre il partizionamento statico, pur pesato sulla cardinalità dei replay, conserva uno sbilanciamento residuo tra le partizioni, che limita il guadagno ottenibile aumentando le repliche.

C. Impatto delle condizioni di rete

Le condizioni di rete sono emulate tramite Linux `tc-netem` nel network namespace dei container interessati. Le regole sono selettive e non alterano il traffico di orchestrazione o raccolta delle metriche. Sul primo confine viene rallentato soltanto `edge-8` nelle scritture verso Kafka; sul secondo viene rallentato il Cloud Worker che, in quella run, possiede la source partition 0 nelle pubblicazioni verso il listener Kafka consumato dal Global Aggregator. I due collegamenti non vengono degradati contemporaneamente.

La campagna usa sei Cloud Worker e confronta una baseline con due tipi di degradazione. Per la latenza viene aggiunto un ritardo di 50 ms. Un pilot con ritardo maggiore aveva saturato la coda di ingresso dell'Edge e introdotto drop a monte; è stato quindi escluso perché non avrebbe isolato l'effetto del watermark. Per la banda sono predisposti limiti pari a 150 kbit/s sul collegamento Edge-Kafka e 100 kbit/s su quello Cloud-Kafka-Global, da applicare separatamente.

Ogni degradazione confronta due politiche di avanzamento. La configurazione *conservative* usa uno skew di 720 ore in event time e attende quindi, di fatto, la sorgente rallentata. La configurazione *bounded* ammette uno skew di 30 minuti in event time, equivalente a 90 ms reali con accelerazione 20 000, e può ridurre il tempo di emissione scartando i contributi che arrivano dopo il superamento della frontiera. Il confronto misura pertanto un compromesso esplicito tra latenza e completezza, non un miglioramento ottenuto nascondendo perdite infrastrutturali.

D. Discussione dei risultati

La campagna di scalabilità mostra che separare partizioni logiche ed esecutori permette di ridurre la latenza di emissione senza cambiare workload o risultati. Il guadagno non è lineare: è limitato dal numero di partizioni, dallo sbilanciamento residuo del loro carico, dal costo del coordinamento tramite Kafka e dalla condivisione delle risorse fisiche. La campagna di rete estende questa analisi distinguendo la politica conservativa, che privilegia la completezza, da quella bounded, che limita l'attesa imponendo un costo misurabile in contributi late.

VI. LIMITAZIONI E DISCUSSIONE

A. Persistenza e guasti

La persistenza del log Kafka non implica il ripristino dello stato delle finestre. Worker e Global mantengono gli aggregati in RAM; i Worker committano in batch soltanto

offset già elaborati, ma non esiste un checkpoint coordinato fra offset e stato. Un crash può quindi perdere contributi già consumati e committati, che non verrebbero riletti al riavvio. Non è garantita una semantica exactly-once end-to-end. Il sink PostgreSQL finale non è uno state store degli aggregatori.

Per questo motivo un rebalance dopo l'inizio dell'elaborazione invalida la run; il worker rifiuta inoltre la ripartenza da offset già committati. Le repliche devono essere pronte prima del replay e restare stabili fino al termine. Un errore richiede una nuova esecuzione completa con topic e offset puliti. Il broker Kafka è singolo, con fattore di replica uno: gli acknowledgment richiesti non realizzano alta disponibilità. La tolleranza ai guasti non è il requisito individuale selezionato.

B. Completezza, carico e rappresentatività

QoS 0 e code limitate possono perdere telemetria. Il protocollo EOS certifica il completamento del flusso accettato dai componenti, non il recupero degli eventi persi a monte. Le run senza perdite devono quindi essere identificate tramite riconciliazione dei contatori e controlli sui risultati. Negli esperimenti di saturazione o rete degradata le perdite sono a loro volta un risultato da riportare; un throughput apparentemente stabile ottenuto scartando dati non soddisfa il requisito di scalabilità.

Le frontiere del Cloud e del Global coincidono con il minimo progresso delle rispettive sorgenti finché il loro disallineamento resta entro il limite configurato. Questa politica preserva la completezza, ma può trattenere finestre in memoria quando una sorgente non avanza. La componente bounded permette di superare una sorgente eccessivamente in ritardo; in tal caso i contributi successivi sono esplicitamente registrati come late e scartati. Un timeout del runner interrompe una prova incompleta, ma non modifica da solo il progresso event-time. Non sono implementati checkpoint, migrazione dello stato o una politica di esclusione dinamica delle sorgenti.

Il workload bilancia i sensori per zona e riguarda un solo tipo di dispositivo in un solo mese. Le medie sono pesate per numero di campioni validi: sensori più frequenti contribuiscono maggiormente. I risultati non possono essere generalizzati automaticamente ad altri dataset, algoritmi più costosi o installazioni geograficamente distribuite. Inoltre, la condivisione degli host e i limiti per container introducono contesa; la scalabilità delle repliche entro un host non dimostra un'illimitata capacità di espansione.

VII. CONCLUSIONI

L'applicazione realizza una pipeline Edge-Cloud in Go con validazione locale, aggregazione gerarchica e comunicazione persistente nei passaggi Kafka. La separazione fra partizioni logiche ed esecutori mantiene invariato il significato degli output al variare delle repliche. Progresso event-time esplicito ed EOS permettono di distinguere un contributo non ancora arrivato da uno effettivamente assente. Con skew ampio le frontiere attendono tutte le sorgenti; con skew limitato forniscono invece un compromesso controllato tra latenza e completezza, rendendo osservabili i contributi scartati come late.

La campagna AWS con due repliche per ciascuna configurazione mostra che il parallelismo Cloud riduce la latenza di emissione mantenendo invariati workload e risultati: passando da uno a quattro worker la mediana diminuisce di circa il 40% e il p95 di circa il 32%. Il passaggio a sei worker non produce ulteriori miglioramenti, evidenziando rendimenti decrescenti dovuti a sbilanciamento, coordinamento e risorse condivise. La scalabilità è quindi dimostrata entro il deployment e il workload analizzati, senza generalizzarla a un'espansione illimitata. La campagna di rete è strutturata per separare gli effetti della degradazione del collegamento da quelli della politica bounded, mantenendo espliciti i limiti dello stato volatile e della telemetria con possibili perdite.

RIFERIMENTI BIBLIOGRAFICI

- [1] V. Cardellini, *Progetto A2: Applicazione Distribuita nel Compute Continuum*, Sistemi Distribuiti e Cloud Computing, Università degli Studi di Roma Tor Vergata, A.A. 2025/26, pp. 1–3.
- [2] Sensor.Community, *Get historical datas*, documentazione della comunità sull'archivio delle osservazioni. <https://forum.sensor.community/t/get-historical-datas/1648>. Consultato l'11 settembre 2026.
- [3] Apache Software Foundation, *Apache Kafka – Design*, documentazione 4.3. <https://kafka.apache.org/43/design/design/>. Consultato l'11 settembre 2026.
- [4] Apache Software Foundation, *Apache Avro 1.12.0 – Specification*. <https://avro.apache.org/docs/1.12.0/specification/>. Consultato l'11 settembre 2026.